NETWORK MONITORING

The network tab in your browser provides tools for monitoring network activity, allowing you to track API calls and data-fetching patterns and identify potential bottlenecks in network requests. HTTP proxies can capture traffic between server and client, providing better visibility into data transfer. It's best to combine this with the structured logging to correlate important network events like request/response data or status.

MEMORY PROFILING

Memory leaks can become significant factors when using streaming or manipulating long lists of messages in memory without proper cleanup procedures. For instance, you might inadvertently create dangling references through forgotten timers or callbacks while trying to update AI message thread histories. This can lead to increased memory consumption and slower application performance, especially when message histories become very elongated. You may experience this with ChatGPT, for example, which becomes really slow when your message thread becomes large (some providers like Claude just impose a limit to the thread window). Utilize built-in browser memory-profiling tools to identify memory leaks, track component memory usage, and optimize resource allocation. Regularly profiling your application helps ensure that it remains responsive over time.

With memory profiling, problems can also occur on the server side (Node.js). One helpful facility the Node ecosystem offers is the ability to create a heap profile. Start your Next.js build with the following command:

```
node --heap-prof node_modules/next/dist/bin/next dev
```

This will create a `.heapprofile` file at the end of the build, which you can load into Chrome DevTools Memory tab for analysis. Figure 8.4 is a screenshot of the profile, which shows the memory usage per function.

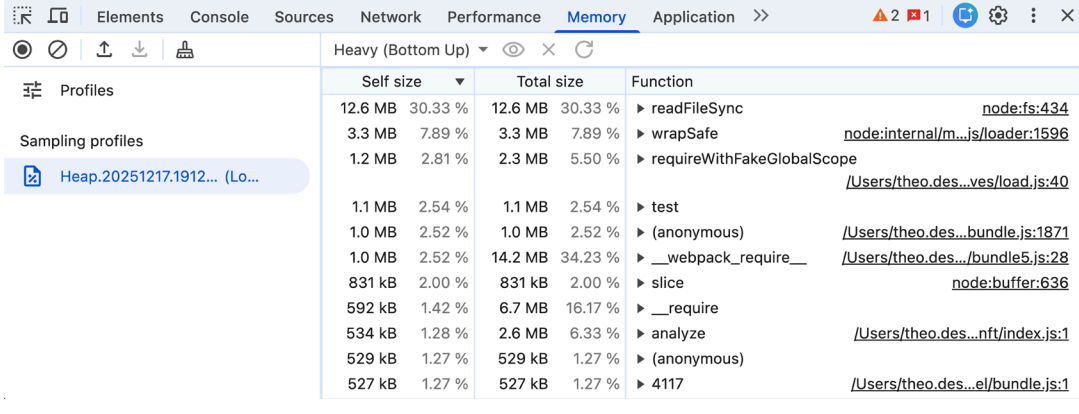


Figure 8.4 Chrome DevTools Memory Profiler view with a heap snapshot. It displays memory allocation statistics for various functions and components in a Node.js application.